

OaldresPuzzle_Cryptic Type-2 Block Cipher Technical Details Paper

English version based on the current /OOP/BlockCipher/ implementation

Twilight-Dream Sparkle-Magic

2026-06-10

Abstract

We propose and formalize the OaldresPuzzle_Cryptic Type-2 stateful block-cipher construction. The normative object of this paper is the current /OOP/BlockCipher/ implementation. The construction connects master-key words, an initialization vector, matrix state, a generated round-subkey matrix, a reversible word-level Lai-Massey round, a state-dependent F function, and a static byte-substitution layer into one end-to-end encryption and decryption path. We do not design the construction merely by increasing the key size, the plaintext space, the ciphertext space, or the number of rounds. We design it so that classical differential, linear, algebraic, related-key, and state-recovery analysis, as well as quantum-oracle circuit construction, must all face a coupled system of prime-field arithmetic, Boolean bit-vector semantics, modular word arithmetic, stateful matrix trajectories, data-dependent lookup, self-masking, and carry propagation.

The main-subkey hardening layer is defined by

$$H_M(x) = Mx + \text{TDOM-Sponge}(Mx) \pmod{p}, \quad p = 2^{64} - 59.$$

Here $Mx \pmod{p}$ is the SIS/Ajtai-form matrix image over the prime field $\mathbb{F}_p$, while TDOM-Sponge is a white-box sponge specified by public round constants, public index and rotation schedules, and a χ -like Boolean layer. It is not treated as a black-box random oracle. The hardened material is then passed through a 32-bit expansion layer, matrix initialization, NLFSR/SDP-driven state update, and a Kronecker outer-product narrow injection to form A_t, T_t, G_t, π_t . The round-subkey layer uses a non-commutative matrix expression, key-state whitening, and a bit-uniform projection $M_{\text{bit}} = D \otimes I_{64}$. The data round uses CrazyTransformAssociatedWord inside a Lai-Massey word permutation; this F function combines matrix addressing, bit extraction, mask deletion, two-channel subkey mixing, and addition in $\mathbb{Z}/2^{32}\mathbb{Z}$.

We give implementation semantics, mathematical definitions, detailed breakable algorithms, and locally provable properties: the collision, differential, and linear constraints of the SIS/Ajtai core; the absorb-permute-squeeze path of the white-box sponge; the collapsed outer-product form $T = A(\delta v u) = \delta(Av)u$ of the Kronecker injection; the row and column degree of the Bit-Matrix; the single-bit fan-out property; the invertibility of the PHT and Lai-Massey word permutation; the forward and inverse byte-substitution tables; and the $16 \times N^2$ macro-round schedule. We do not claim a complete IND-CPA or PRP reduction, and we do not reduce the entire block cipher to SIS or LWE. The paper instead decomposes the construction into auditable mathematical objects and explains how those objects raise the modeling cost of classical analysis and the resource cost of quantum attack circuits. The relevant background includes SIS/Ajtai hashing, sponge constructions, S-box analysis, Lai-Massey structures, and quantum attack models[16, 17, 8, 10, 18, 27, 4, 29, 7, 2, 5, 6, 22, 21, 20, 19, 26, 11, 12, 9, 24, 28, 1, 25, 14, 13, 3, 15, 23].

Keywords: OaldresPuzzle_Cryptic; Type-2 block cipher; SIS/Ajtai hash; Montgomery prime field; white-box sponge; Lai-Massey; BitMatrix; stateful key schedule; post-quantum symmetric cryptography.

Contents

1 Overview: Complexity Design, Quantum Gate-Circuit Cost, and Resistance to Classical Analysis	5
1.1 Key space, plaintext space, and ciphertext space	5
1.2 Real-world quantum-engineering target	6
1.3 Basic operations as quantum gates	6
2 Research Object, Algorithm Boundary, and Exposition Principles	7
3 Overall Structure, Global Pseudocode, and Complexity Motivation	7
4 Abbreviations, Mathematical Notation, and Operational Semantics	8
5 Common State Object: Modules_OaldresPuzzle_Cryptic.hpp	9
6 Montgomery Prime-Field Layer: ExtraIncludes/MontgomeryScalarEigen.hpp	9
7 Custom Sponge Hash: CustomSecureHash.hpp	10
8 Why We Send the SIS/LWE Space into the White-Box Sponge	11
8.1 Kronecker outer-product injection	13
9 Round-Subkey Matrix, Whitening, and BitMatrix: Module_SecureRoundSubkeyGeneratation.*	13
10 PHT and Lai–Massey Invertibility	14
11 State-Dependent F Function: CrazyTransformAssociatedWord	15
12 Static Byte-Substitution Layer	15
13 Data-Block Round Function: OaldresPuzzle_Cryptic.cpp	16
14 Worker Layer and Core Scope: OPC_MainAlgorithm_Worker.*	17
15 PRNG State Material and Internal CSPRNG Subsystems	17
15.1 LFSR	17
15.2 NLFSR	17
15.3 SDP-CSPRNG	18
16 Detailed Breakable Algorithms for Core Subroutines	18
17 Module-wise Formal Expansion and Non-compressed Implementation Correspondence	21
17.1 CommonStateData and primitive boundary: CommonStateData	21
17.2 Montgomery scalar and SIS/Ajtai field image: MontgomeryScalarEigen	22
17.3 TDOM white-box sponge state machine: CustomSecureHash	23
17.4 Secure subkey generation: Module_SecureSubkeyGeneratation	23
17.5 32-bit mix transformation utilities: Module_MixTransformationUtil	24
17.6 Subkey matrix initialization and state update: Module_SubkeyMatrixOperation	25
17.7 Kronecker outer-product injection: SubkeyMatrixOperation/Kronecker	25
17.8 Round-subkey matrix transformation: Module_SecureRoundSubkeyGeneratation	26
17.9 Key-state whitening: GenerationRoundSubkeys whitening	27

17.10 BitMatrix projection: GenerationRoundSubkeys BitMatrix	27
17.11 PHT forward and inverse: ForwardTransform/BackwardTransform	28
17.12 Lai-Massey word permutation: LaiMasseyFramework	28
17.13 CrazyTransformAssociatedWord: CrazyTransformAssociatedWord	29
17.14 Static byte substitution: ByteSubstitution	30
17.15 RoundFunction macro schedule: RoundFunction	30
17.16 Worker stateful scheduling: OPC_MainAlgorithm_Worker	31
17.17 LFSR source: LinearFeedbackShiftRegister	32
17.18 NLFSR source: NonlinearFeedbackShiftRegister	32
17.19 SDP source: SimulateDoublePendulum	33
18 Non-compressed Code-to-Math Alignment for the Current Implementation	34
18.1 Parameter checking and state allocation	34
18.2 Key-word loading and field encoding	34
18.3 Prime-field image and kernel geometry	35
18.4 TDOM displacement and twist graph	35
18.5 Word expansion dependency graph	36
18.6 Matrix fill order	36
18.7 IV and state-material separation	37
18.8 LFSR material consumption	37
18.9 NLFSR feedback slots	38
18.10 SDP sampled material	38
18.11 Affine-broadcast matrix mutation	39
18.12 Outer-product bottleneck	40
18.13 Non-commutative round-key expression	40
18.14 Whitening as state composition	41
18.15 BitMatrix as bit-vector projection	41
18.16 Round-subkey vector lifetime	42
18.17 PHT arithmetic domain	42
18.18 F-function row and column selection	43
18.19 Mask deletion channel	43
18.20 Carry injection in F	44
18.21 Byte substitution table boundary	44
18.22 Macro-round nesting	45
18.23 Worker-level padding boundary	45
18.24 Reversible quantum embedding	46
18.25 Classical analysis interface	46
19 Concrete Default-Parameter Resource Ledger	47
20 Proof and Analysis Ledgers for Non-compressed Review	47
20.1 SIS image collision ledger	47
20.2 Linear mask ledger	48

20.3	Sponge reproducibility ledger	48
20.4	32-bit expansion degree ledger	48
20.5	Matrix-state trajectory ledger	48
20.6	Outer-product narrowness ledger	49
20.7	Round-key non-commutativity ledger	49
20.8	Whitening ledger	49
20.9	BitMatrix proof ledger	49
20.10	CrazyTransform lookup ledger	49
20.11	Mask-channel ledger	50
20.12	Carry ledger	50
20.13	Lai-Massey correctness ledger	50
20.14	PHT inverse ledger	50
20.15	Byte-substitution ledger	50
20.16	RoundFunction work ledger	51
20.17	Worker state ledger	51
20.18	LFSR ledger	51
20.19	NLFSR ledger	51
20.20	SDP ledger	52
20.21	Quantum oracle ledger	52
20.22	Grover cost ledger	52
20.23	Simon model ledger	52
20.24	Differential-analysis ledger	52
20.25	Linear-analysis ledger	53
21	End-to-End Correctness and Implementation–Mathematics Correspondence	53
22	Design Motivation, Analysis Boundary, and Future Work	53
23	Conclusion	53

1 Overview: Complexity Design, Quantum Gate-Circuit Cost, and Resistance to Classical Analysis

We place the complexity design of Type-2 OPC at the beginning because the construction is not an ordinary block cipher with a few complicated accessories. We design it against two kinds of adversaries. The first kind uses classical differential, linear, algebraic, related-key, and state-recovery methods. The second kind tries to embed the complete algorithm into a reversible quantum oracle or a related-key query framework, and then attempts Grover-type search, Simon-type period finding, quantum collision search, or quantum related-key analysis. We therefore make the cost of a single oracle query part of the design target, rather than treating the round function in isolation.

We know that a complicated cipher can easily be accused of being “complex for complexity’s sake.” Our answer is not a slogan. We make every layer a named and auditable mathematical object. The SIS/Ajtai prime-field core constrains master-subkey differences by short linear relations. The TDOM white-box sponge sends the prime-field image through a public but nonlinear state permutation. The 32-bit expansion layer breaks local word structure. The matrix-state trajectory prevents the round material from being a static table. The Kronecker outer-product injection compresses SDP-generated material into a directional state injection. The non-commutative matrix expression propagates state paths. The key-state whitening composes old and new round material. The BitMatrix layer distributes parity uniformly at the bit level. The state-dependent F function places matrix access, masking, subkey mixing, and carry propagation inside the Lai–Massey update. This is a duty chain from key material to data rounds, not a pile of unrelated decorations.

1.1 Key space, plaintext space, and ciphertext space

Let $B = \text{OPC_QuadWord_DataBlockSize}$ and $K = \text{OPC_QuadWord_KeyBlockSize}$. Both are measured in 64-bit quadwords. The implementation requires $B \geq 2$, B even, $K \geq 4$, $K \equiv 0 \pmod{4}$, $K > B$, and $K \equiv 0 \pmod{B}$. A single plaintext block and a single ciphertext block have length $64B$ bits, and one master-key block has length $64K$ bits. Therefore

$$\mathcal{P}_B = \mathcal{C}_B = (\mathbb{Z}/2^{64}\mathbb{Z})^B, \quad \mathcal{K}_K = (\mathbb{Z}/2^{64}\mathbb{Z})^K,$$

with cardinalities

$$|\mathcal{P}_B| = |\mathcal{C}_B| = 2^{64B}, \quad |\mathcal{K}_K| = 2^{64K}.$$

If a padded and packed message contains m data blocks, then the message-level space is $(\mathbb{Z}/2^{64}\mathbb{Z})^{mB}$. If the external caller supplies r master-key blocks, then the raw key-material space is $(\mathbb{Z}/2^{64}\mathbb{Z})^{rK}$ and has size 2^{64Kr} . This notation distinguishes the key block processed by one key-schedule step from the variable-length external key material.

In the current C API and tests the default parameters are $B = 16$ and $K = 32$, hence $N = 2K = 64$. The default block size is 1024 bits and the default master-key block size is 2048 bits. The default core spaces are

$$|\mathcal{P}_{16}| = |\mathcal{C}_{16}| = 2^{1024}, \quad |\mathcal{K}_{32}| = 2^{2048}.$$

Table 1: Default Type-2 OPC parameter scale

Parameter	Symbol	Default	Unit	Implementation source
Data block size	B	16	64-bit words	OPC_QuadWord_DataBlockSize
Master-key block size	K	32	64-bit words	OPC_QuadWord_KeyBlockSize
Matrix dimension	N	64	rows/columns	$N = 2K$
Plaintext block	$64B$	1024	bits	block definition
Ciphertext block	$64B$	1024	bits	block definition
Master-key block	$64K$	2048	bits	key material
Round-subkey vector	N^2	4096	64-bit words	$\text{vec}(G_t)$
Macro-rounds	R	16	rounds	RoundFunction
Word updates per block	$16N^2$	65536	LM updates	default instance

The minimum legal parameter instance is $B = 2$, $K = 4$, giving a 128-bit block and a 256-bit master-key block. The default instance used throughout the main analysis is much larger: a 1024-bit data block and a 2048-bit master-key block. We do not fold the IV, LFSR seed, NLFSR seed, or SDP seed into the master-key space. Those objects are stateful parameters or initialization-state material, because the implementation assigns them different roles.

1.2 Real-world quantum-engineering target

We do not present an absolute defense against a mythical theoretical quantum computer with unbounded fault-tolerant depth, free large-scale qRAM, free reversible memory, free uncomputation, and direct access to an ideal encryption oracle. In such a model no concrete engineering construction can use implementation complexity as an absolute barrier. The target considered here is closer to an engineered fault-tolerant quantum attack. An adversary must implement the construction as a reversible circuit and pay for Toffoli gates, T gates, CNOT networks, controlled rotations, reversible adders, reversible multipliers, controlled table selection, matrix access, uncomputation, and error correction.

Classical symmetric ciphers are often strengthened by increasing the key length, the block size, or the number of rounds. Those methods are still meaningful against Grover search because a 2^κ search is reduced to roughly $2^{\kappa/2}$ queries, but key length remains a first resource factor. Type-2 OPC does not stop at increasing κ . We also increase the cost of each oracle query. Every Grover iteration must expand the key schedule, update matrix state, generate round subkeys, access data-dependent matrix entries, execute Lai–Massey word updates, and then uncompute intermediate state to obtain a clean oracle.

A conventional encryption oracle is

$$U_E : |K, X, Y\rangle \mapsto |K, X, Y \oplus E_K(X)\rangle.$$

For Type-2 OPC a clean oracle must include the initial state and the generated trajectory:

$$U_{\text{OPC}} : |K, IV, X, Y, S_0, 0^a\rangle \mapsto |K, IV, X, Y \oplus E_{K,IV,S_0}(X), S_t, G_t, 0^{a'}\rangle.$$

To obtain a clean query the adversary must implement a compute-copy-uncompute pattern,

$$U_{\text{clean}} \simeq U_{KS}^\dagger U_{\text{Round}} U_{KS},$$

so the key-schedule state cannot simply be thrown away. If $C_Q(F)$ denotes the reversible implementation cost of a classical map F , then

$$\begin{aligned} C_Q(\text{OPC}) &\geq C_Q(H_M) + C_Q(\text{Expand}_{32}) + C_Q(\text{InitState}) \\ &\quad + C_Q(\text{UpdateState}) + C_Q(\text{GenRoundSubkeys}) \\ &\quad + 16N^2 C_Q(\text{LM} \circ F) + 16C_Q(\text{ByteSubstitution}). \end{aligned}$$

1.3 Basic operations as quantum gates

We measure a reversible implementation by a resource vector

$$C_Q(F) = (N_X, N_{\text{CNOT}}, N_{\text{Tof}}, N_T, D_T, A_{\text{clean}}, A_{\text{dirty}}),$$

where the entries count Pauli-X gates, CNOT gates, Toffoli gates, T gates, T-depth, clean ancilla, and dirty ancilla. The following table fixes the gate-level semantics used in the paper.

Table 2: Reversible quantum semantics of Type-2 OPC operations

Classical operation	Reversible implementation semantics	Effect on attack circuit
$x \oplus y$	bitwise CNOT network; overwrite requires preservation of reversible inputs or output registers	linear but wide at scale
$\sim x$	bitwise Pauli-X	cheap alone; often appears with nonlinear gates
$x \& y$	Toffoli target update $t \leftarrow t \oplus xy$	basic source of quadratic Boolean cost
$x y$	NOT plus Toffoli via $x y = \sim((\sim x) \& (\sim y))$	Toffoli-rich Boolean operation
fixed rotation	wire permutation	cheap but changes dependency geometry
data-dependent rotation	controlled barrel shifter	control network and ancilla pressure
addition in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$	reversible adder with carry chain	T-depth and carry propagation

Classical operation	Reversible implementation semantics	Effect on attack circuit
subtraction	reversible adder with two's-complement logic	same carry-chain cost
word multiplication	partial products plus reversible adders	Toffoli and T-count dominated
Montgomery multiplication mod p	reversible multiplication plus conditional reduction	SIS/Ajtai cost core
static S-box	reversible lookup or combinational circuit	table or logic cost
data-dependent matrix entry	controlled select or multiplexer over matrix entries	qRAM-less access disaster
BitMatrix projection	wide CNOT parity network	large linear volume

For the default $K = 32, N = 64$, the matrix state contains $N^2 = 4096$ words. One 64×64 matrix of 64-bit words contains 262144 logical bits. The three matrices A_t, T_t, G_t contain at least 786432 logical bits before auxiliary registers. A matrix-vector multiplication over $\mathbb{F}_p$ contains at least 4096 Montgomery multiplications plus accumulations. A dense $N \times N$ word matrix multiplication contains $N^3 = 262144$ word multiply-add positions. One encrypted block invokes $16N^2 = 65536$ Lai–Massey word updates. This table is why we describe the key schedule as an engineered pain point for quantum circuit construction rather than as a cosmetic complication.

2 Research Object, Algorithm Boundary, and Exposition Principles

The research object is the Type-2 block-cipher core implemented under `/00P/BlockCipher/`. The current core is not the template version, not the stream-cipher branch, and not the external password-processing wrapper. We treat external KDF or password processing as an invocation layer. The primitive studied here begins from 64-bit master-key words and state parameters and ends at the encryption or decryption of packed 64-bit data blocks.

We use the word “core” in a strict sense. The core includes the common state object, the prime-field hardening layer, the white-box sponge used in that hardening layer, the 32-bit expansion functions, matrix initialization, IV injection, state update, round-subkey generation, key-state whitening, BitMatrix projection, the state-dependent F function, the PHT/Lai–Massey word permutation, the static byte-substitution layer, and the stateful worker path that calls those pieces. The core excludes unrelated legacy experiments, template versions, stream-cipher modules, and external password-based key derivation.

The exposition follows the rule: implementation object, design rationale, mathematical definition, local proof, and composition role. We use “we” as the design perspective. The word does not mark a conversation; it marks the design authorship of the construction. Whenever the paper explains a choice, it states what we design, why we design it, what mathematical object it becomes, and what it can or cannot prove by itself.

3 Overall Structure, Global Pseudocode, and Complexity Motivation

The Type-2 core is a stateful block-cipher framework. Its chain is

$$K \longrightarrow H_M(x) \longrightarrow \text{Expand}_{32} \longrightarrow (A_t, T_t, G_t, \pi_t) \longrightarrow g_t \longrightarrow \text{LM}/F/\text{ByteSubstitution}.$$

The key schedule is not an appendix to the data round. It is part of the primitive. The data round receives round material produced by a state trajectory, and the cost of generating that trajectory matters to both classical and quantum analysis.

Algorithm 1 End-to-end Type-2 core encryption path

Require: master-key words K , initialization vector IV , packed data block sequence $X_0, \dots, X_{m-1}$

- 1: Initialize `CommonStateData` with (B, K, N) and initial state containers.
- 2: Convert key words into the prime-field input vector $x \in \mathbb{F}_p^N$.
- 3: Generate public or state-derived matrix $M \in \mathbb{F}_p^{N \times N}$ according to the implementation path.
- 4: Compute $y \leftarrow Mx \pmod{p}$.
- 5: Compute $z \leftarrow y + \text{TDOM-Sponge}(y) \pmod{p}$.
- 6: Expand z through the 32-bit expansion layer.
- 7: Fill the state matrix A_0 and transformed-subkey matrix T_0 .
- 8: Inject IV if the invocation uses IV as state material.

```

9: for  $j = 0$  to  $m - 1$  do
10:   Update matrix state using LFSR/NLFSR/SDP material.
11:   Perform Kronecker outer-product injection.
12:   Generate  $G_t$  by the non-commutative matrix transformation.
13:   Whiten the round-subkey vector with the new  $G_t$  material.
14:   Apply  $M_{\text{bit}} = D \otimes I_{64}$  to the round-subkey bit vector.
15:   Encrypt data block  $X_j$  by 16 macro-rounds and  $N^2$  word updates per macro-round.
16: end for
17: return ciphertext blocks.

```

Algorithm 2 End-to-end Type-2 core decryption path

Require: same master-key words, same initialization state, ciphertext blocks $C_0, \dots, C_{m-1}$

```

1: Reconstruct the same state trajectory and round-subkey sequence as in encryption.
2: for each ciphertext block in the same state order do
3:   for macro-round index in reverse order do
4:     Apply inverse byte substitution.
5:     Apply Lai–Massey word updates in reverse round-key order.
6:   end for
7: end for
8: Unpack and remove padding at the worker layer.
9: return plaintext blocks.

```

The construction is intentionally more than a static SPN. We do not rely on one S-box layer or one linear layer to carry the entire analysis burden. The key material is first placed in a prime-field matrix image, then twisted by a white-box sponge, then expanded into matrices, then evolved through state generators and outer-product injection, then transformed into round-subkey material, and only then consumed by a data round. Each layer has a local responsibility and a local analysis boundary.

4 Abbreviations, Mathematical Notation, and Operational Semantics

Table 3: Abbreviations used in the paper

Abbreviation	Meaning
OPC	OaldresPuzzle_Cryptic
TDOM	Twilight-Dream Oaldres-Mixing white-box sponge
SIS	Short Integer Solution
ISIS	Inhomogeneous Short Integer Solution
LWE	Learning With Errors
PHT	pseudo-Hadamard transform
LM	Lai–Massey construction
PRNG	pseudorandom number generator
CSPRNG	cryptographically oriented pseudorandom generator
LFSR	linear feedback shift register
NLFSR	nonlinear feedback shift register
SDP	simulated double pendulum generator
SAC	strict avalanche criterion
DDT	difference distribution table
LAT	linear approximation table

We use $\mathbb{F}_2 = \mathbb{F}_2$ for bit-vector linear algebra, $\mathbb{Z}/2^{32}\mathbb{Z} = \mathbb{Z}/2^{32}\mathbb{Z}$ and $\mathbb{Z}/2^{64}\mathbb{Z} = \mathbb{Z}/2^{64}\mathbb{Z}$ for unsigned word arithmetic, and $\mathbb{F}_p = \mathbb{F}_p$ for the Montgomery prime field with $p = 2^{64} - 59$. We write $\oplus$ for XOR, $\&$ for bitwise AND, $|$ for bitwise OR, $\sim$ for bitwise NOT, and rotr , rotr for rotations. The notation A_t, T_t, G_t denotes the random/state matrix, transformed-subkey matrix, and generated-round-subkey matrix at time t . The shuffled matrix index schedule is π_t .

Table 4: Algebraic spaces and operational semantics

Operation family	Space	Meaning in the implementation
XOR, AND, OR, NOT	bit-vector Boolean algebra	bitwise operations over 32-bit and 64-bit words
BitMatrix projection	$\mathbb{F}_2$	$M_{\text{bit}} = D \otimes I_{64}$ over flattened bit vectors
32-bit addition/subtraction	$\mathbb{Z}/2^{32}\mathbb{Z}$	unsigned wraparound arithmetic in the F function and PHT layer
64-bit addition/subtraction	$\mathbb{Z}/2^{64}\mathbb{Z}$	unsigned wraparound matrix-state arithmetic
Prime-field matrix-vector multiplication	$\mathbb{F}_p$	SIS/Ajtai-form hardening core
Montgomery multiplication	$\mathbb{F}_p$	constant-time field multiplication modulo p
Matrix multiplication in state schedule	$\mathbb{Z}/2^{64}\mathbb{Z}$ word arithmetic	implementation-level unsigned matrix arithmetic, not $\mathbb{F}_2(2^{64})$

We do not identify every 64-bit matrix multiplication with multiplication in $\mathbb{F}_2(2^{64})$. The only prime-field layer is the Montgomery/SIS-Ajtai hardening component. The BitMatrix layer is over $\mathbb{F}_2$ after flattening 32 words into 2048 bit coordinates. This separation is essential: otherwise the paper would mix incompatible algebraic semantics and would make false claims about the implementation.

5 Common State Object: Modules_OaldresPuzzle_Cryptic.hpp

The common state object stores the block size B , the key-block size K , the matrix dimension $N = 2K$, the IV material, the LFSR/NLFSR/SDP generator state, and the matrices A_t, T_t, G_t . The construction uses this object to make the cipher stateful. A block is not encrypted only under a static sequence of round keys; it is encrypted under a trajectory whose state has been built from the master key, the IV, the PRNG material, and the matrix schedule.

We design this layer to separate primitive parameters from call-layer formatting. B and K are not inferred from byte strings after the fact; they determine the vector and matrix sizes that every following layer uses. The state object therefore provides a single source of truth for $N = 2K$, for the size of G_t , for the length of the round-subkey vector, and for the interpretation of the data block.

Definition 5.1 (Type-2 state object). A Type-2 state object at time t is

$$S_t = (A_t, T_t, G_t, \pi_t, \text{LFSR}_t, \text{NLFSR}_t, \text{SDP}_t, IV),$$

where $A_t, T_t, G_t \in (\mathbb{Z}/2^{64}\mathbb{Z})^{N \times N}$ and π_t is a permutation-like index schedule used by state-dependent matrix lookup.

Proposition 5.1 (state dimension). For the default $K = 32$, $N = 64$, so each matrix contains 4096 64-bit words. The three main matrices contain at least $3 \cdot 4096 \cdot 64 = 786432$ logical bits before any reversible-computation ancilla.

Proof. The definition gives $N = 2K = 64$. A matrix has $N^2 = 4096$ entries. Each entry is a 64-bit word. The three matrices A_t, T_t, G_t therefore contain $3N^2 \cdot 64$ bits. Substituting $N = 64$ gives 786432 bits. $\square$

6 Montgomery Prime-Field Layer: ExtraIncludes/MontgomeryScalarEigen.hpp

The SIS/Ajtai-form hardening layer requires arithmetic in a prime field. The implementation uses the prime

$$p = 18446744073709551557 = 2^{64} - 59.$$

This choice keeps elements close to 64-bit machine words while preserving field arithmetic. Montgomery representation is used to implement multiplication and reduction efficiently.

Let $M \in \mathbb{F}_p^{N \times N}$ and $x \in \mathbb{F}_p^N$. The core matrix image is

$$y = Mx \pmod{p}.$$

The collision equation of the bare map is

$$Mx = Mx' \pmod{p} \iff M(x - x') = 0 \pmod{p}.$$

When $x - x'$ is required to be short in an integer embedding, this is the standard SIS/Ajtai collision geometry. We use this structure as a hardening core for master-subkey material. We do not claim that every later layer is reduced to SIS; the claim is that the first structured hardening step is SIS/Ajtai-form.

Algorithm 3 Montgomery prime-field matrix-vector multiplication

Require: $M \in \mathbb{F}_p^{N \times N}, x \in \mathbb{F}_p^N$

```

1: for  $i = 0$  to  $N - 1$  do
2:    $y_i \leftarrow 0$ 
3:   for  $j = 0$  to  $N - 1$  do
4:      $t \leftarrow \text{MontMul}_p(M_{i,j}, x_j)$ 
5:      $y_i \leftarrow y_i + t \pmod{p}$ 
6:   end for
7: end for
8: return  $y$ 
```

For the default $N = 64$, this computation contains at least 4096 Montgomery multiplications and at least 4032 prime-field additions. In a reversible circuit, the products, partial sums, conditional reductions, and uncomputation all have to be accounted for.

7 Custom Sponge Hash: CustomSecureHash.hpp

The custom sponge is not a black box. It is a white-box state machine whose constants, schedules, and update equations are part of the public construction. The file `SecureCustomHash.md` specifies the derivation of the round constants from a 4096-bit prime and square-root fractional digits; the source contains the resulting tables. Any implementation that uses different constants or schedules is not the TDOM sponge defined in this paper.

Let the sponge state be a vector $S \in (\mathbb{Z}/2^{64}\mathbb{Z})^w$. The message is padded according to the implementation semantics, split into rate blocks, absorbed by XOR/additive mixing into the state, permuted by $\mathcal{P}_h$, and squeezed into output words. The permutation is decomposed into adjacent word mixing, half-state rotation diffusion, index/rotation schedule assignment, a χ -like Boolean layer, and round-constant injection.

We write the χ -like layer abstractly as

$$S'_i = S_i \oplus ((\sim S_{i+1}) \& S_{i+2}) \oplus c_i,$$

where the actual indices and rotations are drawn from fixed public schedules. This expression is quadratic over $\mathbb{F}_2$ because of the AND term. It is cheap for the designer to compute classically, but it adds Toffoli-rich nonlinear gates to a reversible quantum circuit.

Algorithm 4 White-box TDOM sponge

Require: byte string M , output length ℓ

```

1: Pad  $M$  using the implementation padding rule.
2: Split the padded message into rate blocks  $B_0, \dots, B_{r-1}$ .
3: Initialize state  $S \leftarrow S_0$ .
4: for each rate block  $B_i$  do
5:   Absorb  $B_i$  into the rate part of  $S$ .
6:    $S \leftarrow \mathcal{P}_h(S)$ .
7: end for
8: Initialize output buffer  $Z \leftarrow \emptyset$ .
9: while  $|Z| < \ell$  do
10:  Append the rate part of  $S$  to  $Z$ .
11:   $S \leftarrow \mathcal{P}_h(S)$  if more output is required.
12: end while
```

13: **return** first ℓ bytes of Z .

Algorithm 5 One TDOM state permutation

Require: state $S = (S_0, \dots, S_{w-1})$, public schedules I, R , round constants C

```

1: for adjacent word pairs do
2:   Mix neighboring words by XOR, rotation, and addition.
3: end for
4: for half-state positions do
5:   Rotate and diffuse between the left and right halves of the state.
6: end for
7: for scheduled positions  $(i, j, k)$  from  $I$  and rotations from  $R$  do
8:   Apply a  $\chi$ -like update using  $S_i, S_j, S_k$ .
9: end for
10: Inject round constants at both ends of the state.
11: return updated state.

```

The sponge is used as a deterministic nonlinear displacement field on the SIS/Ajtai image. We do not model it as an ideal random oracle. We instead expose its structure and state that its purpose is to harden the bare prime-field image by making differential and linear reasoning depend on the base point inside $\text{Im}(M)$.

8 Why We Send the SIS/LWE Space into the White-Box Sponge

We formulate the design motivation of this layer as a problem of spatial composition. Suppose a high-dimensional discrete lattice-point space is produced from the SIS/Ajtai and LWE setting. What happens if the definition of that space, together with its discrete lattice image, is fed into a hash-like white-box symmetric mixing layer? Going one step further, if the input first falls into the original lattice structure and is then stirred again in the style of a symmetric primitive, does the output remain an ordinary linear image, or does it become a high-dimensional discrete image space twisted by a deterministic nonlinear displacement field? This is the intuition behind the construction: the forward direction is easy for the designer because it consists of matrix multiplication, a white-box sponge, and modular addition; the analysis direction is difficult because the adversary must simultaneously reason in the discrete lattice, the prime-field linear image, the word-parallel Boolean sponge state machine, modular-addition perturbations, and the resulting twisted image space.

We first write the SIS/Ajtai space as the bare skeleton. Let

$$q = p = 2^{64} - 59, \quad M \in \mathbb{Z}_q^{N \times N}, \quad x \in \mathbb{Z}_q^N,$$

and define the Ajtai-form matrix hash core

$$h_M(x) = Mx \bmod q.$$

If the input is restricted to a bounded integer box

$$\mathcal{B}_\beta = \{x \in \mathbb{Z}^N : \|x\|_\infty \leq \beta\},$$

then the collision relation of the bare core is

$$h_M(x) = h_M(x') \iff M(x - x') \equiv 0 \pmod{q}.$$

Hence we obtain the q -ary orthogonal lattice

$$\Lambda_q^\perp(M) = \{z \in \mathbb{Z}^N : Mz \equiv 0 \pmod{q}\},$$

where

$$z = x - x' \in \Lambda_q^\perp(M)$$

is a short vector controlled by the input bound. In other words, the bare SIS/Ajtai layer writes collision search as short-kernel-vector search in a q -ary lattice. This object is clean, linear, and nameable; it is also precisely the analysis coordinate that an adversary would like to preserve. As long as the problem remains only Mx , differential and linear reasoning can be pulled back to the kernel, image, and transpose spaces of M .

We also use LWE as the spatial reference for a linear image perturbed by errors. In the ordinary LWE setting, for

$$A \in \mathbb{Z}_q^{m \times n}, \quad s \in \mathbb{Z}_q^n,$$

one observes

$$b = As + e \pmod{q},$$

where $e \leftarrow \chi^m$ is sampled from an error distribution. The set of visible points can be viewed as

$$\mathcal{C}_{A,\chi}(s) = \{As + e \pmod{q} : e \in \text{supp}(\chi)^m\}.$$

This is not the same object as the Type-2 hardening layer, and the paper does not claim an LWE reduction for the block cipher. The role of LWE here is to clarify the spatial intuition: a linear image may become significantly harder to analyze when it is not viewed alone but is accompanied by a perturbation field. In LWE that perturbation is sampled noise; in Type-2 OPC it is a deterministic, public, white-box sponge displacement driven by the prime-field image itself.

We define the prime-field image

$$\mathcal{L}_M = \text{Im}(M) = \{Mx : x \in \mathbb{Z}_q^N\} \subseteq \mathbb{Z}_q^N.$$

The TDOM sponge induces a displacement field on this image. Let ϕ encode a prime-field vector into the sponge input words, and let ψ decode the sponge output words back into $\mathbb{Z}_q^N$. We define

$$S_h(y) = \psi(\text{TDOM-Sponge}_h(\phi(y))).$$

The twist map is

$$\tau_h(y) = y + S_h(y) \pmod{q}.$$

Therefore the actual hardening function is

$$H_M(x) = \tau_h(Mx) = Mx + S_h(Mx) \pmod{q}.$$

The output space is no longer the bare linear image $\mathcal{L}_M$; it is the twisted image

$$\tau_h(\mathcal{L}_M) = \{y + S_h(y) : y \in \mathcal{L}_M\},$$

and, when the original lattice image and the displaced image are observed together, the graph space

$$\Gamma_{M,h} = \{(y, \tau_h(y)) : y \in \mathcal{L}_M\} = \{(y, y + S_h(y)) : y \in \mathcal{L}_M\}.$$

This is the precise mathematical expression of the “strange space” intuition: the first coordinate keeps the auditable SIS/Ajtai skeleton, while the second coordinate moves that skeleton through a deterministic white-box symmetric mixing field.

We fold differential difficulty directly into this twisted image. Given an input difference Δx , let

$$y = Mx, \quad \Delta y = M\Delta x \pmod{q}.$$

For the bare SIS/Ajtai core,

$$\Delta h_M = h_M(x + \Delta x) - h_M(x) = \Delta y.$$

A zero difference in the bare core only asks for

$$M\Delta x = 0 \pmod{q},$$

which is exactly the homogeneous short relation $\Delta x \in \Lambda_q^\perp(M)$. After the image is sent into the white-box sponge, the differential is

$$\begin{aligned} \Delta H_M(x; \Delta x) &= H_M(x + \Delta x) - H_M(x) \\ &= M\Delta x + S_h(y + M\Delta x) - S_h(y) \pmod{q}. \end{aligned}$$

Define

$$\Delta_S(y, \Delta y) = S_h(y + \Delta y) - S_h(y).$$

Then

$$\Delta H_M = \Delta y + \Delta_S(y, \Delta y) \pmod{q}.$$

Thus the zero-difference condition is changed from the fixed homogeneous target

$$\Delta y = 0$$

to the moving inhomogeneous target

$$\Delta y = -\Delta_S(y, \Delta y) \pmod{q}.$$

Equivalently,

$$M\Delta x = -(S_h(y + M\Delta x) - S_h(y)) \pmod{q}.$$

The adversary is no longer solving only a short-kernel problem. They must track a base-point-dependent nonlinear displacement on $\text{Im}(M)$.

The same phenomenon appears in linear analysis. For a linear mask α , the bare prime-field image gives

$$\langle \alpha, h_M(x) \rangle = \langle M^T \alpha, x \rangle \pmod{q}.$$

After the TDOM displacement is added,

$$\langle \alpha, H_M(x) \rangle = \langle M^T \alpha, x \rangle + \langle \alpha, S_h(Mx) \rangle \pmod{q}.$$

The first term is the ordinary transpose mask of the bare matrix core. The second term is a mask induced by the white-box sponge on the image $\text{Im}(M)$. It is not a fixed linear coordinate in x . Therefore a low-complexity mask cannot be evaluated by looking only at $M^T \alpha$; it must also account for the image-dependent sponge displacement.

This construction does not assert that τ_h preserves standard SIS hardness for every possible M , nor does it assert that H_M inherits an LWE reduction. The claim is narrower and more concrete: if an adversary tries to build short differences or low-complexity masks along the bare SIS/Ajtai core, they must also satisfy the nonlinear displacement-cancellation conditions induced by TDOM-Sponge_{*h*} on $\text{Im}(M)$. The design therefore separates the easy forward path for the constructor from the hard unified coordinate system required by the analyst.

8.1 Kronecker outer-product injection

The Kronecker stage is intentionally narrow. It does not try to generate a full-entropy random matrix directly. It takes two direction vectors and an inner product:

$$K_{\otimes} = vu, \quad \delta = \langle v, u \rangle, \quad T_{t+1} = A_t(\delta vu) = \delta(A_t v)u.$$

In a field embedding, vu has rank at most one. In $\mathbb{Z}/2^{64}\mathbb{Z}$ the ring has zero divisors, so field rank cannot be used naively, but the outer-product bottleneck still exists. This is why we call it a collapsed, outer-product, rank-at-most-one-style narrow map. We design it as a directional injection that is then amplified by the subsequent non-commutative matrix expression, whitening, BitMatrix projection, and state-dependent F function.

Algorithm 6 Matrix state update with Kronecker injection

Require: matrices A_t, T_t , generator material u, v

- 1: Build affine-broadcast matrices L and R from A_t, u, v .
 - 2: **for** each matrix position (i, j) **do**
 - 3: $\alpha_{i,j} \leftarrow L_{i,j} \oplus (A_{i,j} \& T_{i,j})$
 - 4: $\beta_{i,j} \leftarrow R_{i,j} \oplus (A_{i,j} | T_{i,j})$
 - 5: $A_{i,j} \leftarrow A_{i,j} \oplus (\text{rotr}(\alpha_{i,j}, 1) + \text{rotl}(\beta_{i,j}, 63))$
 - 6: **end for**
 - 7: $K_{\otimes} \leftarrow vu$
 - 8: $\delta \leftarrow \langle v, u \rangle$
 - 9: $T_{t+1} \leftarrow A_t(\delta K_{\otimes})$
 - 10: **return** updated (A_{t+1}, T_{t+1}) .
-

9 Round-Subkey Matrix, Whitening, and BitMatrix:

Module_SecureRoundSubkeyGeneratation.*

The round-subkey layer transforms the matrix state into the round material consumed by the data round. Its matrix expression is

$$G_{t+1} = G_t + (T_t^T - A_t)(A_t^T + T_t)A_tT_t.$$

The products are implementation-level 64-bit word arithmetic, not multiplication in $\mathbb{F}_2(2^{64})$. The expression is non-commutative: changing the order changes the result. This path sensitivity is one reason the generated round material is difficult to isolate as a static key table.

Key-state whitening then combines the previous round-subkey vector with the newly generated matrix entries. This is key-state whitening, not plaintext whitening. It is applied before the BitMatrix projection so that old and new round material are mixed at the vector level.

The BitMatrix layer is specified by a 32-by-32 incidence matrix D whose rows and columns each have degree 16. The implementation stores it as a hard-coded XOR ledger: each output word is the XOR of 16 input words. Because each word is 64 bits, the actual bit-level map is

$$\text{bit}(Y) = (D \otimes I_{64}) \text{bit}(X).$$

This is not a word-level MDS matrix claim. It is a bit-uniform parity distribution layer. A single input bit in one word lane fans out to 16 output bits in the same lane across different output words, and each output bit is the parity of 16 input bits.

Proposition 9.1 (row and column degree of the BitMatrix). If every row and every column of D has Hamming weight 16, then every output bit of $D \otimes I_{64}$ depends on 16 input bits, and every input bit contributes to 16 output bits.

Proof. The Kronecker product $D \otimes I_{64}$ replaces each 1 entry in D by a 64-by-64 identity matrix and each 0 entry by a zero matrix. Hence the row and column weights of D lift lane-wise to the bit matrix. A row of weight 16 becomes 16 identity blocks in the corresponding bit rows, and a column of weight 16 becomes 16 identity blocks in the corresponding bit columns. $\square$

Algorithm 7 Round-subkey generation

Require: A_t, T_t, G_t , previous vector g_t

1: $G_{t+1} \leftarrow G_t + (T_t^T - A_t)(A_t^T + T_t)A_tT_t$

2: $h \leftarrow \text{vec}(G_{t+1})$

3: **for** each vector index i **do**

4: $g_i \leftarrow g_i \oplus h_i$

▷ key-state whitening

5: **end for**

6: **for** each 32-word chunk X of g **do**

7: $Y \leftarrow (D \otimes I_{64})X$

8: replace the chunk by Y

9: **end for**

10: **return** transformed round-subkey vector.

10 PHT and Lai–Massey Invertibility

Each 64-bit data word is split into two 32-bit halves, $W = L \parallel R$. The Lai–Massey update computes

$$a = L \oplus R, \quad t = F(a, k), \quad L' = L \oplus t, \quad R' = R \oplus t.$$

The invariant is

$$L' \oplus R' = (L \oplus t) \oplus (R \oplus t) = L \oplus R = a.$$

Therefore the decryptor can recompute the same $F(a, k)$ after inverting the following PHT-like transform. The function F itself does not need to be invertible; it only needs to be deterministic and recomputable from the invariant.

The PHT-like transform is expressed over $\mathbb{Z}/2^{32}\mathbb{Z}$:

$$A = L + R, \quad B = L + 2R,$$

followed by rotation-XOR mixing. Its inverse is derived by undoing the rotation-XOR layer and then solving

$$R = B - A, \quad L = 2A - B$$

in $\mathbb{Z}/2^{32}\mathbb{Z}$. Thus the word update is a permutation even though F is not a permutation.

Algorithm 8 Lai–Massey word encryption

Require: word $W = L \parallel R$, round key k

1: $a \leftarrow L \oplus R$

2: $t \leftarrow F(a, k)$

3: $L_1 \leftarrow L \oplus t, R_1 \leftarrow R \oplus t$

4: $(L_2, R_2) \leftarrow H(L_1, R_1)$

5: **return** $L_2 \parallel R_2$

Algorithm 9 Lai–Massey word decryption

Require: word $W' = L_2 \parallel R_2$, round key k

1: $(L_1, R_1) \leftarrow H^{-1}(L_2, R_2)$

2: $a \leftarrow L_1 \oplus R_1$

3: $t \leftarrow F(a, k)$

4: $L \leftarrow L_1 \oplus t, R \leftarrow R_1 \oplus t$

5: **return** $L \parallel R$

11 State-Dependent F Function: CrazyTransformAssociatedWord

The F function is not an S-box, not a pure ARX function, and not a random lookup. It is a state-dependent, piecewise Boolean/modular map. The input is a 32-bit associated word a and a 64-bit word key k . The key is split into $k_L = \text{hi}_{32}(k)$ and $k_R = \text{lo}_{32}(k)$. A pseudo-random value is formed by combining k , a , and $\sim k$; four 32-bit working words A, B, C, D are reorganized by shifts, XOR, OR, AND, NOT, and rotations. The transformed A, B select row and column positions through the shuffled schedule π_t , and the selected matrix word $q = G_{r,c}$ is used to form a mask.

The function then uses two channels: a masked round-subkey channel $\tilde{q} = q \& \sim m$ and an original round-subkey channel q . The final result is an addition in $\mathbb{Z}/2^{32}\mathbb{Z}$:

$$F(a, k) = a + (T_1(A, B, \tilde{q}) \oplus T_2(C, D, q)) \pmod{2^{32}}.$$

The first channel depends on A, B and the masked halves of q . The second channel depends on C, D and the original halves of q . The final modular addition introduces carry propagation, so a bit-level linear model cannot remain inside pure XOR algebra.

Algorithm 10 State-dependent F function

Require: associated word $a \in \mathbb{Z}/2^{32}\mathbb{Z}$, key word $k \in \mathbb{Z}/2^{64}\mathbb{Z}$, matrix G_t , schedule π_t

- 1: $k_L \leftarrow \text{hi}_{32}(k), k_R \leftarrow \text{lo}_{32}(k)$
 - 2: $P \leftarrow ((k \oplus a) \ll 32) \mid (((\sim k) \oplus a) \gg 32)$
 - 3: Derive C, D by shifts of P .
 - 4: Mix a, k_L, k_R into C, D using XOR, OR, AND, and NOT.
 - 5: Initialize $A \leftarrow C, B \leftarrow D$ and apply rotations using $P \bmod 32$.
 - 6: Feed C, D back into A, B with complemented key halves.
 - 7: $r \leftarrow \pi_t[A \bmod |\pi_t|], c \leftarrow \pi_t[B \bmod |\pi_t|]$
 - 8: $q \leftarrow G_t[r, c]$
 - 9: Extract two bits from q using shifts derived from A, B, r, c .
 - 10: Rotate the two bits into a 64-bit mask m .
 - 11: $\tilde{q} \leftarrow q \& \sim m$
 - 12: Split q and $\tilde{q}$ into 32-bit halves.
 - 13: Compute $T_1 \leftarrow \text{rotl}(A \oplus \text{lo}_{32}(\tilde{q}), 16) \oplus \text{rotr}(B \oplus \text{hi}_{32}(\tilde{q}), 16)$
 - 14: Compute $T_2 \leftarrow \text{hi}_{32}(q) - \text{rotl}(C \oplus \text{lo}_{32}(q), 8) \oplus \text{rotr}(D, 8)$
 - 15: **return** $a + (T_1 \oplus T_2) \pmod{2^{32}}$
-

This map is intentionally pathological. The designer can compute it by following the state and the matrix lookup. The analyst cannot replace it by a single fixed S-box, because row selection, column selection, masking, and the final carry chain all depend on the evolving key-state material.

12 Static Byte-Substitution Layer

The byte-substitution layer is a set of four 8-bit permutations used by a fixed 8-byte schedule on each 64-bit word. It is not the runtime ZUC stream cipher. The current Type-2 core uses static tables and their inverse tables. The forward encryption schedule is

$$(S_{F1}, S_{F0}, S_{B1}, S_{B0}, S_{F0}, S_{B1}, S_{F0}, S_{B1}),$$

and the inverse schedule is

$$(S_{B1}, S_{B0}, S_{F1}, S_{F0}, S_{B0}, S_{F1}, S_{B0}, S_{F1}).$$

The inverse tables satisfy

$$S_{B0}(y) = x \iff S_{F0}(x) = y, \quad S_{B1}(y) = x \iff S_{F1}(x) = y.$$

The first forward table is generated over $\mathbb{F}_2(2^8)$ using the primitive polynomial

$$x^8 + x^5 + x^4 + x^3 + x^2 + x + 1,$$

then an inversion-affine transformation

$$S_{F0}(x) = \text{vec}^{-1}(A_0 \text{vec}(\text{Inv}_0(x)) \oplus c_0).$$

This explains why $S_{F0}(0)$ is determined by the affine constant. The second forward table is a ZUC-style static table used here only as a table; Type-2 core does not execute ZUC keystream generation, ZUC LFSR, bit reorganization, or linear transform.

Algorithm 11 ByteSubstitution forward schedule

Require: 64-bit word W split into bytes $b_0, \dots, b_7$

- 1: $b'_0 \leftarrow S_{F1}(b_0)$
 - 2: $b'_1 \leftarrow S_{F0}(b_1)$
 - 3: $b'_2 \leftarrow S_{B1}(b_2)$
 - 4: $b'_3 \leftarrow S_{B0}(b_3)$
 - 5: $b'_4 \leftarrow S_{F0}(b_4)$
 - 6: $b'_5 \leftarrow S_{B1}(b_5)$
 - 7: $b'_6 \leftarrow S_{F0}(b_6)$
 - 8: $b'_7 \leftarrow S_{B1}(b_7)$
 - 9: **return** packed word $W' = b'_0 \parallel \dots \parallel b'_7$
-

Algorithm 12 ByteSubstitution inverse schedule

Require: substituted word $W' = b'_0 \parallel \dots \parallel b'_7$

- 1: Apply the inverse table schedule $(S_{B1}, S_{B0}, S_{F1}, S_{F0}, S_{B0}, S_{F1}, S_{B0}, S_{F1})$.
 - 2: Pack the recovered bytes into W .
 - 3: **return** W .
-

S-box metrics are computed by standard tables. For a vectorial S-box S , the difference distribution table is

$$\text{DDT}[a, b] = |\{x : S(x \oplus a) \oplus S(x) = b\}|,$$

and the linear approximation table is

$$\text{LAT}[a, b] = \sum_x (-1)^{\langle a, x \rangle \oplus \langle b, S(x) \rangle}.$$

The differential uniformity is $\delta = \max_{a \neq 0, b} \text{DDT}[a, b]$, and component nonlinearity is computed by

$$\text{NL}(S_i) = 2^7 - \frac{1}{2} \max_a |\text{LAT}[a, e_i]|.$$

Thus the S-box metric table is not a free assertion; it is derived from DDT, LAT, SAC, transparency-order, propagation, robustness, algebraic degree, and algebraic-immunity definitions.

13 Data-Block Round Function: OaldresPuzzle_Cryptic.cpp

Let $X = (x_0, \dots, x_{B-1}) \in (\mathbb{Z}/2^{64}\mathbb{Z})^B$ be a block and let $g = (g_0, \dots, g_{N^2-1})$ be the transformed round-subkey vector. One encrypted block performs 16 macro-rounds. In each macro-round, all N^2 round-subkey words are consumed; the target data word is selected by $j = i \bmod B$.

Algorithm 13 RoundFunction encryption

Require: data block $X = (x_0, \dots, x_{B-1})$, round-subkey vector g

- 1: **for** $r = 0$ to 15 **do**
 - 2: **for** $i = 0$ to $N^2 - 1$ **do**
 - 3: $j \leftarrow i \bmod B$
 - 4: $x_j \leftarrow \text{LMEnc}(x_j, g_i)$
 - 5: **end for**
 - 6: **for** $j = 0$ to $B - 1$ **do**
 - 7: $x_j \leftarrow \text{ByteSubstitution}(x_j)$
 - 8: **end for**
 - 9: **end for**
 - 10: **return** X
-

Algorithm 14 RoundFunction decryption

Require: ciphertext block X , same round-subkey vector g

- 1: **for** $r = 15$ downto 0 **do**
- 2: **for** $j = 0$ to $B - 1$ **do**
- 3: $x_j \leftarrow \text{InverseByteSubstitution}(x_j)$
- 4: **end for**

```

5:   for  $i = N^2 - 1$  downto 0 do
6:      $j \leftarrow i \bmod B$ 
7:      $x_j \leftarrow \text{LMDec}(x_j, g_i)$ 
8:   end for
9: end for
10: return  $X$ 

```

For the default $N = 64$, a single block performs $16 \cdot 4096 = 65536$ word-level Lai–Massey updates. This is the number that matters for quantum oracle construction, because each update includes the state-dependent F function and must be implemented reversibly if it appears inside a quantum search oracle.

14 Worker Layer and Core Scope: `OPC_MainAlgorithm_Worker.*`

The worker layer packs bytes into 64-bit words, applies padding and unpadding, dispatches blocks through the stateful core, and exposes the encryption/decryption interface. We treat packing and padding as representation and message-format operations rather than as the main security contribution. They are still specified because they determine which data blocks enter the core.

The worker is stateful. A correct test of decryption must reconstruct the same initial state and key schedule trajectory, or store the trajectory required for inverse processing. Reusing a mutated encryption object as if it were still at the initial state would test a different state machine. This is a semantic property of the construction, not an appendix-level engineering detail.

Algorithm 15 Worker-level encryption skeleton

Require: byte string P , master-key words K , initialization state S_0

```

1: Pad  $P$  according to the worker padding rule.
2: Pack padded bytes into blocks in  $(\mathbb{Z}/2^{64}\mathbb{Z})^B$ .
3: Initialize Type-2 core state from  $K$  and  $S_0$ .
4: for each data block do
5:   Generate or update round-subkey state.
6:   Apply RoundFunction encryption.
7: end for
8: Unpack ciphertext words into bytes.
9: return ciphertext byte string.

```

15 PRNG State Material and Internal CSPRNG Subsystems

The LFSR, NLFSR, and SDP components provide state material for the matrix schedule. They are not used as stand-alone security assumptions. Their role is to feed the state update with low-cost linear material, nonlinear gated material, and continuous-system sampled material.

15.1 LFSR

The LFSR has a 128-bit state represented by two 64-bit words. Its feedback polynomial is

$$x^{128} + x^{41} + x^{39} + x + 1.$$

It is linear over $\mathbb{F}_2$, so it is not used as a stand-alone cryptographic generator. It is used as a low-cost state material source whose output is later consumed by nonlinear and matrix layers.

15.2 NLFSR

The NLFSR uses four 64-bit state words and nine feedback-polynomial slots. A 9-bit gate selects active feedback components, and the update uses fixed loops rather than data-dependent loop counts. The combining functions include nonlinear Boolean terms. If f contains an AND term, then its algebraic normal form has degree at least two over $\mathbb{F}_2$. The NLFSR therefore supplies nonlinear state material to the affine-broadcast matrix update.

15.3 SDP-CSPRNG

The SDP generator maps key-derived material into double-pendulum initial conditions, advances a continuous dynamical system by explicit time steps, extracts fractional values, and combines two 32-bit halves by Morton/interleaving into 64-bit words. We use it as a deterministic, key-dependent state material generator. The paper does not claim that a double pendulum alone is a standard CSPRNG proof; it claims that SDP material is one component in a larger state schedule.

Algorithm 16 Division of labor of the internal state-material sources

- 1: LFSR: generate low-cost linear progression material.
 - 2: NLFSR: generate nonlinear gated feedback material.
 - 3: SDP: generate key-dependent continuous-system sampled material.
 - 4: Matrix update: consume all three sources and mix them into A_t, T_t .
 - 5: Round-subkey generation: transform matrix state into G_t and g_t .
-

16 Detailed Breakable Algorithms for Core Subroutines

The following algorithms are not C++ listings. They are the mathematical pseudocode counterparts of the implementation path. They are included in the main body so that no essential algorithmic step is hidden behind a source appendix.

Algorithm 17 Detailed pseudocode: Pack bytes into 64-bit words

- 1: Read the byte stream in little-endian groups of eight bytes.
 - 2: Pad the final group when required by the worker rule.
 - 3: Interpret each group as one element of $\mathbb{Z}/2^{64}\mathbb{Z}$.
 - 4: Collect B words into one data block.
 - 5: Return the block sequence.
-

Algorithm 18 Detailed pseudocode: Unpack 64-bit words into bytes

- 1: For each word, emit its eight bytes in the implementation order.
 - 2: Concatenate bytes from all words in block order.
 - 3: Remove padding only at the worker boundary.
 - 4: Return the recovered byte string.
-

Algorithm 19 Detailed pseudocode: Generate SIS/Ajtai matrix material

- 1: Initialize SDP rejection sampling state.
 - 2: Sample candidate field elements modulo p .
 - 3: Fill the matrix M in row-major order.
 - 4: Reject elements that do not satisfy implementation constraints.
 - 5: Return $M \in \mathbb{F}_p^{N \times N}$.
-

Algorithm 20 Detailed pseudocode: Compute hardening vector

- 1: Convert key words to field vector x .
 - 2: Compute $y = Mx \pmod{p}$.
 - 3: Encode y into TDOM sponge input.
 - 4: Compute $S_h(y)$ by white-box sponge.
 - 5: Return $z = y + S_h(y) \pmod{p}$.
-

Algorithm 21 Detailed pseudocode: Generate TDOM constants

- 1: Start from the public 4096-bit prime.
 - 2: Compute the square-root fractional digit stream.
 - 3: Cut the digit stream into 64-bit chunks.
 - 4: Map chunks to round constants.
 - 5: Publish the constants as part of the TDOM definition.
-

Algorithm 22 Detailed pseudocode: Generate TDOM index schedule

- 1: Seed the public ISAAC64 schedule generator.
- 2: Generate move bit counts.

- 3: Generate hash-state indices.
- 4: Freeze the resulting tables.
- 5: Use the same tables in every conforming implementation.

Algorithm 23 Detailed pseudocode: TDOM absorption step

- 1: Select the rate part of the state.
 - 2: XOR or mix the message block into rate words.
 - 3: Leave capacity words unavailable to direct input.
 - 4: Apply the state permutation.
-

Algorithm 24 Detailed pseudocode: TDOM squeeze step

- 1: Read the rate portion of the state.
 - 2: Append output words to the digest buffer.
 - 3: Permute again if the requested output is not yet filled.
-

Algorithm 25 Detailed pseudocode: WordBitRestruct

- 1: Select bit positions from the input word.
 - 2: Swap or move scheduled bits.
 - 3: Return the restructured 32-bit word.
-

Algorithm 26 Detailed pseudocode: Six-register chi-like expansion

- 1: Initialize six registers from input and key material.
 - 2: Apply XOR and rotation mixing.
 - 3: Apply AND/OR based chi-like updates.
 - 4: Apply PHT-like add/subtract mixing.
 - 5: Emit twelve 32-bit words.
-

Algorithm 27 Detailed pseudocode: Initialize matrix state

- 1: Read expanded 32-bit words.
 - 2: Combine pairs into 64-bit words where required.
 - 3: Fill A_0 and T_0 according to implementation order.
 - 4: Initialize G_0 and the index schedule.
-

Algorithm 28 Detailed pseudocode: Inject initialization vector

- 1: Check that the IV size is compatible with the block/state size.
 - 2: Map IV bytes to 64-bit state words.
 - 3: Mix the IV words into the state matrices.
 - 4: Continue the same state trajectory after injection.
-

Algorithm 29 Detailed pseudocode: LFSR step

- 1: Compute feedback bit from the configured taps.
 - 2: Shift the 128-bit state.
 - 3: Insert the feedback bit.
 - 4: Accumulate output bits into a word.
-

Algorithm 30 Detailed pseudocode: NLFSR feedback selection

- 1: Compute the gate bits.
 - 2: Select active feedback polynomial slots.
 - 3: Update four 64-bit lanes.
 - 4: Apply nonlinear combining functions.
-

Algorithm 31 Detailed pseudocode: SDP initialization

- 1: Split the binary seed into parameter fields.
 - 2: Map fields to angles, velocities, and parameters.
 - 3: Reject invalid or degenerate states.
 - 4: Return the initial double-pendulum state.
-

Algorithm 32 Detailed pseudocode: SDP stepping

- 1: Evaluate double-pendulum equations.
 - 2: Advance the state by the configured time step.
 - 3: Repeat for the configured number of internal steps.
 - 4: Extract fractional coordinates.
-

Algorithm 33 Detailed pseudocode: Kronecker injection

- 1: Generate row vector u and column vector v .
 - 2: Compute $K_{\otimes} = vu$.
 - 3: Compute $\delta = \langle v, u \rangle$.
 - 4: Set $T = A(\delta vu)$.
-

Algorithm 34 Detailed pseudocode: Round-subkey matrix transformation

- 1: Compute $L = T^T - A$.
 - 2: Compute $R = A^T + T$.
 - 3: Compute $U = LR$.
 - 4: Compute $V = AT$.
 - 5: Update $G \leftarrow G + UV$.
-

Algorithm 35 Detailed pseudocode: Key-state whitening

- 1: Vectorize the generated round-subkey matrix.
 - 2: XOR it into the previous round-subkey vector.
 - 3: Keep the operation in key-state space, not data space.
-

Algorithm 36 Detailed pseudocode: BitMatrix projection

- 1: Split the round-subkey vector into 32-word chunks.
 - 2: For each output word, XOR the selected 16 input words.
 - 3: This implements $(D \otimes I_{64})$ lane-wise.
 - 4: Replace the chunk by the projected chunk.
-

Algorithm 37 Detailed pseudocode: CrazyTransform addressing

- 1: Compute A, B, C, D working words.
 - 2: Use A and B to index the shuffled row and column schedules.
 - 3: Read $q = G[r, c]$.
 - 4: Extract selected bits from q .
-

Algorithm 38 Detailed pseudocode: CrazyTransform mask and output

- 1: Rotate selected bits into a mask.
 - 2: Compute $\tilde{q} = q \& \sim m$.
 - 3: Compute the masked channel T_1 .
 - 4: Compute the original-subkey channel T_2 .
 - 5: Return $a + (T_1 \oplus T_2) \pmod{2^{32}}$.
-

Algorithm 39 Detailed pseudocode: PHT forward

- 1: Compute $A = L + R \pmod{2^{32}}$.
 - 2: Compute $B = L + 2R \pmod{2^{32}}$.
 - 3: Apply rotation-XOR mixing.
 - 4: Return transformed halves.
-

Algorithm 40 Detailed pseudocode: PHT backward

- 1: Undo rotation-XOR mixing.
 - 2: Compute $R = B - A \pmod{2^{32}}$.
 - 3: Compute $L = 2A - B \pmod{2^{32}}$.
 - 4: Return original halves.
-

Algorithm 41 Detailed pseudocode: Byte substitution table generation

- 1: Generate S_{F0} by finite-field inversion and affine transform.
- 2: Use the static ZUC-style table as S_{F1} .
- 3: Compute S_{B0} and S_{B1} as inverse tables.
- 4: Freeze all four tables.

Algorithm 42 Detailed pseudocode: S-box metric evaluation

- 1: Build DDT for every nonzero difference.
- 2: Build LAT for every input/output mask pair.
- 3: Compute differential uniformity.
- 4: Compute component nonlinearity and related metrics.

Algorithm 43 Detailed pseudocode: RoundFunction macro-round

- 1: For each of 16 macro-rounds, iterate over all N^2 round-subkey words.
- 2: Update word $i \bmod B$ by Lai–Massey.
- 3: Apply byte substitution to every block word.

Algorithm 44 Detailed pseudocode: RoundFunction inverse macro-round

- 1: For each macro-round in reverse order, first invert byte substitution.
- 2: Then consume round-subkey words in reverse order.
- 3: Apply inverse Lai–Massey word update.

Algorithm 45 Detailed pseudocode: Stateful worker schedule

- 1: Build state from key and IV.
- 2: For each block, update state and generate round material.
- 3: Encrypt or decrypt the block.
- 4: Advance the state trajectory.

Algorithm 46 Detailed pseudocode: Quantum clean oracle construction

- 1: Compute the full key schedule reversibly.
 - 2: Compute the data round reversibly.
 - 3: Copy the ciphertext into the oracle output register.
 - 4: Uncompute round and key-schedule ancilla.
-

17 Module-wise Formal Expansion and Non-compressed Implementation Correspondence

This section expands the implementation path as main-body technical content. It is not a source-code appendix. Each subsection states the code object, the design intention, the mathematical object, the local proof obligation, and the role of the module in the complete construction. The purpose is to make the English version as explicit as the Chinese technical manuscript and to prevent any reader from treating a compressed formula as a hidden black box.

17.1 CommonStateData and primitive boundary: CommonStateData

We identify CommonStateData as an implementation object with a precise mathematical role. The state object is the contract between all modules. It fixes the block size, the key-block size, the derived matrix dimension, the matrices, the IV material, and the internal generator state. The object must be read as an algebraic state, not as a convenience struct. Each member determines a mathematical coordinate of the cipher trajectory. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that

has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 47 Expanded correspondence for CommonStateData and primitive boundary

- 1: Identify the source-level object as CommonStateData.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.2 Montgomery scalar and SIS/Ajtai field image: MontgomeryScalarEigen

We identify MontgomeryScalarEigen as an implementation object with a precise mathematical role. The field layer gives the only prime-field arithmetic domain used by the construction. It supports the map $y=Mx \bmod p$ and supplies the SIS/Ajtai-form skeleton. The design isolates the field domain so that unsigned word matrix arithmetic is never falsely identified with finite-field multiplication. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 48 Expanded correspondence for Montgomery scalar and SIS/Ajtai field image

- 1: Identify the source-level object as MontgomeryScalarEigen.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
-

- 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.3 TDOM white-box sponge state machine: CustomSecureHash

We identify CustomSecureHash as an implementation object with a precise mathematical role. The sponge is a deterministic white-box permutation system. Its constants, index schedules, rotations, and chi-like operations are part of the public algorithm definition. The design turns a linear field image into a twisted image space by adding a deterministic nonlinear displacement field. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 49 Expanded correspondence for TDOM white-box sponge state machine

- 1: Identify the source-level object as CustomSecureHash.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.4 Secure subkey generation: Module_SecureSubkeyGeneratation

We identify Module_SecureSubkeyGeneratation as an implementation object with a precise mathematical role. This module constructs the first hardened key image. It moves from raw key words to a prime-field vector, applies the matrix image, and folds the sponge displacement back into the field. The layer is the place where SIS/Ajtai geometry and symmetric white-box mixing are fused. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module.

The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 50 Expanded correspondence for Secure subkey generation

- 1: Identify the source-level object as `Module_SecureSubkeyGeneratation`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.5 32-bit mix transformation utilities: `Module_MixTransformationUtil`

We identify `Module_MixTransformationUtil` as an implementation object with a precise mathematical role. This module widens and restructures 32-bit words before matrix initialization. It introduces bit restructuring, rotations, Boolean degree growth, and carry-sensitive additions. The layer prevents the matrix state from being a direct formatting of the prime-field digest. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 51 Expanded correspondence for 32-bit mix transformation utilities

- 1: Identify the source-level object as `Module_MixTransformationUtil`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
-

- 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.6 Subkey matrix initialization and state update: Module_SubkeyMatrixOperation

We identify Module_SubkeyMatrixOperation as an implementation object with a precise mathematical role. This module fills A and T, injects IV material, consumes internal generator material, and advances the matrix state. The design treats state evolution as a security-relevant part of the primitive, not as an external schedule. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 52 Expanded correspondence for Subkey matrix initialization and state update

- 1: Identify the source-level object as Module_SubkeyMatrixOperation.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.7 Kronecker outer-product injection: SubkeyMatrixOperation/Kronecker

We identify SubkeyMatrixOperation/Kronecker as an implementation object with a precise mathematical role. The outer product $K = vu$ and scalar $\delta = \langle v, u \rangle$ create $T = A(\delta vu) = \delta(Av)u$. This is a collapsed narrow map rather than full randomization. The narrowness is intentional: it gives a directional injection that is later amplified by non-commutative matrix multiplication and the data-dependent round function. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 53 Expanded correspondence for Kronecker outer-product injection

- 1: Identify the source-level object as SubkeyMatrixOperation/Kronecker.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.8 Round-subkey matrix transformation: Module_SecureRoundSubkeyGeneratation

We identify Module_SecureRoundSubkeyGeneratation as an implementation object with a precise mathematical role. The expression $G \leftarrow G + (T^T - A)(A^T + T)AT$ maps matrix state into generated round material. The ordering matters. The design uses non-commutativity and accumulation to make round material trajectory-dependent. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 54 Expanded correspondence for Round-subkey matrix transformation

- 1: Identify the source-level object as Module_SecureRoundSubkeyGeneratation.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.9 Key-state whitening: GenerationRoundSubkeys whitening

We identify `GenerationRoundSubkeys whitening` as an implementation object with a precise mathematical role. The vectorized generated matrix is XORed into the round-subkey vector before `BitMatrix` projection. This is key-state whitening, not plaintext whitening; it combines old and new key-state material. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 55 Expanded correspondence for Key-state whitening

- 1: Identify the source-level object as `GenerationRoundSubkeys whitening`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.10 BitMatrix projection: GenerationRoundSubkeys BitMatrix

We identify `GenerationRoundSubkeys BitMatrix` as an implementation object with a precise mathematical role. The hard-coded XOR ledger is a compressed representation of $D \otimes I_{64}$. It acts on the flattened bit vector. The design target is bit-uniform parity distribution, not a word-level MDS claim. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-

commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 56 Expanded correspondence for BitMatrix projection

- 1: Identify the source-level object as `GenerationRoundSubkeys BitMatrix`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.11 PHT forward and inverse: ForwardTransform/BackwardTransform

We identify `ForwardTransform/BackwardTransform` as an implementation object with a precise mathematical role. The PHT-like layer mixes two 32-bit halves and has an explicit inverse over $\mathbb{Z}/2^{32}\mathbb{Z}$. This layer supplies reversible local diffusion after the Lai-Massey F injection. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 57 Expanded correspondence for PHT forward and inverse

- 1: Identify the source-level object as `ForwardTransform/BackwardTransform`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.12 Lai-Massey word permutation: LaiMasseyFramework

We identify `LaiMasseyFramework` as an implementation object with a precise mathematical role. The invariant $L \text{ xor } R$ is preserved across the F injection, so F need not be invertible. The design uses a non-invertible-looking state-dependent F while

keeping the word-level permutation invertible. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 58 Expanded correspondence for Lai-Massey word permutation

- 1: Identify the source-level object as `LaiMasseyFramework`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.13 **CrazyTransformAssociatedWord:** CrazyTransformAssociatedWord

We identify `CrazyTransformAssociatedWord` as an implementation object with a precise mathematical role. The function reorganizes four working words, uses key halves, selects a matrix entry, masks it, and returns a modular-additive output. The design makes the F function a state-dependent, piecewise Boolean/modular map with matrix lookup and carry injection. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle.

XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 59 Expanded correspondence for CrazyTransformAssociatedWord

- 1: Identify the source-level object as CrazyTransformAssociatedWord.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.14 Static byte substitution: ByteSubstitution

We identify ByteSubstitution as an implementation object with a precise mathematical role. The layer applies four 8-bit permutations with a fixed per-byte schedule and inverse schedule. The design adds byte-level confusion at macro-round boundaries without invoking a runtime stream cipher. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 60 Expanded correspondence for Static byte substitution

- 1: Identify the source-level object as ByteSubstitution.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.15 RoundFunction macro schedule: RoundFunction

We identify RoundFunction as an implementation object with a precise mathematical role. Each block executes sixteen macro-rounds, and each macro-round consumes N^2 round-subkey words. The design makes a single block expensive: default parameters produce 65536 word updates per block. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 61 Expanded correspondence for RoundFunction macro schedule

- 1: Identify the source-level object as RoundFunction.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.16 Worker stateful scheduling: OPC_MainAlgorithm_Worker

We identify OPC_MainAlgorithm_Worker as an implementation object with a precise mathematical role. The worker packs bytes, applies padding, drives stateful round generation, and unpacks output. The worker is a boundary layer; it should not be confused with external KDF or password processing. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 62 Expanded correspondence for Worker stateful scheduling

- 1: Identify the source-level object as `OPC_MainAlgorithm_Worker`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.17 LFSR source: LinearFeedbackShiftRegister

We identify `LinearFeedbackShiftRegister` as an implementation object with a precise mathematical role. The LFSR is a linear low-cost state-material source. It is not a stand-alone security assumption; later nonlinear and matrix layers consume its output. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F , and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 63 Expanded correspondence for LFSR source

- 1: Identify the source-level object as `LinearFeedbackShiftRegister`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.18 NLFSR source: NonlinearFeedbackShiftRegister

We identify `NonlinearFeedbackShiftRegister` as an implementation object with a precise mathematical role. The NLFSR uses gated feedback slots and nonlinear combining functions. It provides nonlinear state material to break purely linear generator behavior. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 64 Expanded correspondence for NLFSR source

- 1: Identify the source-level object as `NonlinearFeedbackShiftRegister`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
 - 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

17.19 SDP source: `SimulateDoublePendulum`

We identify `SimulateDoublePendulum` as an implementation object with a precise mathematical role. The SDP generator maps seed material to a double-pendulum system and extracts fractional state information. It supplies deterministic continuous-system-derived state material, not a stand-alone proof of randomness. We use the module in the construction only through the semantics described here; we do not rely on an unstated implementation convention.

The input space, output space, and state dependency are specified before the formula is used. If the module consumes a word vector, the word vector belongs to either $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_p$ as stated by the section. If the module consumes a flattened bit vector, the operation is over $\mathbb{F}_2$. This prevents an analyst from silently moving a formula from one algebraic space into another.

The design rationale is that each local transformation changes the coordinate system available to the adversary. A differential condition that is simple before the module is transformed into a different condition after the module. A linear mask that has a single algebraic meaning before the module may split into a field term, a Boolean term, and a carry term after the module. The module is therefore not justified by appearance; it is justified by the way it changes the equations available to differential, linear, algebraic, and quantum-circuit analysis.

The local proof obligation is deliberately limited. We prove size, domain, determinism, invertibility when the module is supposed to be reversible, and row/column degree when the module is a linear bit projection. We do not claim that any single module proves the security of the whole cipher. The full construction obtains its analysis resistance from composition: a prime-field skeleton, a white-box nonlinear displacement, word expansion, matrix state, narrow outer-product injection, non-commutative round-key generation, key-state whitening, bit-uniform projection, state-dependent F, and reversible data-round closure.

For quantum analysis, the module must be translated into a reversible circuit if it participates in a clean encryption oracle. XOR contributes CNOT volume, AND and OR contribute Toffoli and T gates, additions contribute carry chains, table access contributes controlled selection, and matrix operations contribute many repeated word-level arithmetic subcircuits. The fact that a module is efficient for the designer in classical C++ does not mean that it is cheap for a fault-tolerant quantum adversary.

Algorithm 65 Expanded correspondence for SDP source

- 1: Identify the source-level object as `SimulateDoublePendulum`.
 - 2: Declare its input domain and output domain before using it in a formula.
 - 3: Translate each word operation into its algebraic space.
-

- 4: Translate each bit operation into Boolean algebra over $\mathbb{F}_2$.
 - 5: State whether the module is reversible, deterministic, or state-evolving.
 - 6: State the local property used by the surrounding construction.
 - 7: Compose the local property with the preceding and following modules.
-

18 Non-compressed Code-to-Math Alignment for the Current Implementation

This section exists in the main body because the English version must not compress implementation semantics into a black box. Each item gives the reader the exact reason that a source-level step has a mathematical role. The goal is not to paste C++ code, but to expose the equation that the code implements and the analysis question that the equation creates.

18.1 Parameter checking and state allocation

The constructor checks the block and key dimensions before any round material is generated. This check is not cosmetic: it fixes the dimensions of every matrix, vector, chunk, and loop bound used later.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 66 Non-compressed alignment procedure: Parameter checking and state allocation

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.2 Key-word loading and field encoding

The key material enters as 64-bit words and is interpreted as field elements only at the SIS/Ajtai hardening layer. The same bit string must not be silently reinterpreted as a byte password, a stream-cipher seed, or a word-level mask.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism,

invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 67 Non-compressed alignment procedure: Key-word loading and field encoding

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.3 Prime-field image and kernel geometry

The matrix image produces a high-dimensional linear object over the prime field. The kernel of this map is the place where bare SIS-style collisions would have to live.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 68 Non-compressed alignment procedure: Prime-field image and kernel geometry

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.4 TDOM displacement and twist graph

The white-box sponge induces a deterministic displacement on the field image. The resulting graph records both the original image point and the displaced point, so an adversary has to model the map and the displacement together.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 69 Non-compressed alignment procedure: TDOM displacement and twist graph

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.5 Word expansion dependency graph

The 32-bit expansion layer should be viewed as a dependency graph whose vertices are words and whose edges are bit extraction, rotation, Boolean mixing, and carry propagation.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 70 Non-compressed alignment procedure: Word expansion dependency graph

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.6 Matrix fill order

Matrix initialization determines how expanded words become coordinates of A and T. Changing the fill order changes the state trajectory, so it is part of the definition.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 71 Non-compressed alignment procedure: Matrix fill order

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.7 IV and state-material separation

The IV is state material. It changes the trajectory, but it is not the same object as the master-key block and is not counted as master-key space unless explicitly made secret by a protocol.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 72 Non-compressed alignment procedure: IV and state-material separation

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.8 LFSR material consumption

The LFSR output is linear over $\mathbb{F}_2$. It remains useful because it is not consumed as a final keystream; it is mixed into nonlinear and matrix layers.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound

arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 73 Non-compressed alignment procedure: LFSR material consumption

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.9 NLFSR feedback slots

The NLFSR provides gated nonlinearity. The gate selection prevents the state update from being a single fixed linear recurrence.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 74 Non-compressed alignment procedure: NLFSR feedback slots

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.10 SDP sampled material

The double-pendulum generator gives deterministic continuous-system-derived words. The construction does not rely on chaos as a proof; it uses the material as another state coordinate.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 75 Non-compressed alignment procedure: SDP sampled material

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.11 Affine-broadcast matrix mutation

The affine-broadcast update creates row-wise and column-wise dependence. Each matrix entry sees a local state word and a broadcast generator word.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 76 Non-compressed alignment procedure: Affine-broadcast matrix mutation

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.12 Outer-product bottleneck

The Kronecker injection deliberately creates a directional narrow map. Its value is not full entropy; its value is making the following matrix expression respond to a low-rank directional stimulus.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 77 Non-compressed alignment procedure: Outer-product bottleneck

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.13 Non-commutative round-key expression

The order of matrix factors is part of the design. The expression is not reducible to a commutative product without changing the implementation.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 78 Non-compressed alignment procedure: Non-commutative round-key expression

- 1: Declare the source-level operation and its input objects.
- 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
- 3: Write the mathematical transformation without omitting state variables.
- 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
- 5: State the local property used by the next layer.
- 6: State the classical analysis equation changed by this layer.

18.14 Whitening as state composition

Whitening composes previous and current key-state material. It is not intended as a final security layer but as a way to prevent newly generated matrix material from replacing old state material in a simple overwrite.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 79 Non-compressed alignment procedure: Whitening as state composition

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.15 BitMatrix as bit-vector projection

The hard-coded XOR lists are not merely word lists. They are the compressed form of a 2048-by-2048 binary matrix acting lane-wise.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 80 Non-compressed alignment procedure: BitMatrix as bit-vector projection

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
-

- 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.16 Round-subkey vector lifetime

The generated vector is used by the round function and then the state moves on. Analysis must account for this lifetime, not just for one vector in isolation.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 81 Non-compressed alignment procedure: Round-subkey vector lifetime

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.17 PHT arithmetic domain

The PHT inverse is valid over $\mathbb{Z}/2^{32}\mathbb{Z}$. Its proof would be wrong if the additions were treated as XOR or as prime-field addition.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 82 Non-compressed alignment procedure: PHT arithmetic domain

- 1: Declare the source-level operation and its input objects.
- 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.

- 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.18 F-function row and column selection

The F function turns internal words into indices. This creates data-dependent access to the round-subkey matrix and forces controlled selection in a quantum circuit.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 83 Non-compressed alignment procedure: F-function row and column selection

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.19 Mask deletion channel

The masked subkey and the original subkey are both used. The two-channel structure prevents a single simplified subkey variable from representing the whole F function.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 84 Non-compressed alignment procedure: Mask deletion channel

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.20 Carry injection in F

The final addition in R32 is a carry chain. Differential and linear models cannot keep the function purely in F2 after this point.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 85 Non-compressed alignment procedure: Carry injection in F

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.21 Byte substitution table boundary

The byte layer uses static tables and their inverses. The use of a ZUC-style table as a static table does not import ZUC runtime semantics into the Type-2 core.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 86 Non-compressed alignment procedure: Byte substitution table boundary

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.22 Macro-round nesting

The outer count is 16, but the inner count is N^2 . The true work per block is therefore macro-rounds times round-subkey vector length.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 87 Non-compressed alignment procedure: Macro-round nesting

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.23 Worker-level padding boundary

Padding and packing are necessary for representing messages, but they are not the cryptographic innovation. They define the interface to the block primitive.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 88 Non-compressed alignment procedure: Worker-level padding boundary

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.24 Reversible quantum embedding

Every intermediate value used by a quantum oracle must either remain available or be uncomputed. This makes the state trajectory cost visible.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 89 Non-compressed alignment procedure: Reversible quantum embedding

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

18.25 Classical analysis interface

Classical analysis sees several coupled equations: field equations, Boolean equations, word-carry equations, and state-index equations. None of them alone is the complete object.

We identify the input object, the output object, and the algebraic domain before giving the transformation. This discipline is important because the implementation moves repeatedly between $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, and $\mathbb{F}_2$. When an operation is in $\mathbb{F}_p$, collision equations and linear masks have prime-field meaning. When an operation is in $\mathbb{Z}/2^{32}\mathbb{Z}$ or $\mathbb{Z}/2^{64}\mathbb{Z}$, carries and wraparound arithmetic become part of the analysis. When an operation is in $\mathbb{F}_2$, row degree, column degree, and parity propagation become the relevant objects.

The design reason is not hidden. We use this step to change the coordinates that an adversary would prefer to keep simple. A bare linear relation is made base-point dependent; a word relation becomes bit-level and carry-sensitive; a static round-key table becomes a state trajectory; a cheap classical lookup becomes a controlled selection problem in a reversible circuit.

The local claim is deliberately limited. This step contributes a specific property to the whole construction, but it does not prove the security of the whole construction alone. The proof obligation is to show domain correctness, determinism, invertibility if required, and the exact dependency that the next layer receives.

For a quantum adversary, the corresponding reversible circuit must preserve enough information to uncompute temporary registers. Therefore the cost of this step is not only the forward classical cost; it includes copied outputs, ancilla pressure, T-depth for nonlinear gates, CNOT volume for parity networks, and controlled selection cost for tables or matrices.

Algorithm 90 Non-compressed alignment procedure: Classical analysis interface

- 1: Declare the source-level operation and its input objects.
 - 2: Assign every input coordinate to $\mathbb{F}_p$, $\mathbb{Z}/2^{32}\mathbb{Z}$, $\mathbb{Z}/2^{64}\mathbb{Z}$, or $\mathbb{F}_2$.
 - 3: Write the mathematical transformation without omitting state variables.
 - 4: Identify whether the transformation is reversible, deterministic, narrowing, or state-evolving.
 - 5: State the local property used by the next layer.
 - 6: State the classical analysis equation changed by this layer.
 - 7: State the quantum-circuit resource class introduced by this layer.
-

19 Concrete Default-Parameter Resource Ledger

The default parameters $B = 16$, $K = 32$, $N = 64$ are used to make the cost discussion concrete. One block contains 1024 data bits. One master-key block contains 2048 key bits. The round-subkey matrix has 4096 64-bit entries. A single macro-round consumes 4096 state-dependent word keys, and one encrypted block consumes 65536 Lai–Massey word updates.

Table 5: Default resource quantities relevant to classical and quantum analysis

Quantity	Default value	Analysis role
Data block size	1024 bits	size of plaintext/ciphertext block space
Master-key block size	2048 bits	Grover baseline and key-schedule input
Matrix dimension	64	dimension of A, T, G, M
Matrix entries	4096 words	state and round-subkey scale
One matrix bit size	262144 bits	reversible state register size
Three matrix bit size	786432 bits	minimal A, T, G state size
Prime-field GEMV	4096 multiplications	SIS/Ajtai hardening cost
Dense matrix multiplication	262144 word multiply-add positions	round-key matrix transformation cost
BitMatrix chunk	30720 XOR compositions	CNOT parity volume per 32-word chunk
Lai–Massey updates per block	65536	F-function and reversible round cost

These numbers do not prove security by themselves. They make the design target concrete: the attacker does not obtain a cheap round oracle after paying only for a key length. The attacker must also pay for state generation, matrix arithmetic, bit projection, data-dependent addressing, and uncomputation.

20 Proof and Analysis Ledgers for Non-compressed Review

This section records proof-level ledgers that would otherwise be scattered across module descriptions. They are part of the main body. They are included to make explicit what each component proves locally, what it does not prove alone, and which analysis equation it changes.

20.1 SIS image collision ledger

The bare map $x \mapsto Mx$ gives the equation $M\Delta x = 0$. The twisted map gives $M\Delta x + \Delta S_h = 0$. This is the algebraic reason the sponge is not a decorative post-processing step.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.2 Linear mask ledger

For a mask α , the expression $\langle \alpha, H_M(x) \rangle$ splits into $\langle M^T \alpha, x \rangle$ and the sponge-induced term. The latter prevents the mask from remaining a single linear coordinate.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.3 Sponge reproducibility ledger

The constants and schedules of TDOM are not implementation secrets. A conforming implementation must use the same public constants and fixed schedules; otherwise it implements a different function.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.4 32-bit expansion degree ledger

Boolean AND terms increase algebraic degree. Modular addition introduces carry dependencies. Rotations move dependencies across bit positions. The output dependency is therefore not a simple word copy.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.5 Matrix-state trajectory ledger

The state matrices are updated over time. An analyst cannot replace A_t, T_t, G_t by one fixed matrix without losing the stateful semantics of the cipher.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.6 Outer-product narrowness ledger

The map $T = A(\delta vu)$ is narrow by construction. It is not a random-matrix sampler. Its purpose is to inject a direction into a later non-commutative matrix expression.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.7 Round-key non-commutativity ledger

The products $(T^T - A)(A^T + T)AT$ are ordered. Reordering them changes the result, hence the expression creates path sensitivity.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.8 Whitening ledger

Whitening joins the previous vector and the generated matrix vector. It is not plaintext whitening and must not be analyzed as if it were an XOR of data with a key.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.9 BitMatrix proof ledger

Because $D \otimes I_{64}$ replaces every one in D by a 64-bit identity block, row and column degree lift exactly from word-incidence to bit-incidence.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.10 CrazyTransform lookup ledger

The matrix entry read by the F function is selected by transformed data and key material. In a reversible circuit this is a controlled selection, not a free array read.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.11 Mask-channel ledger

The F function uses both q and $\tilde{q} = q \& \sim m$. A simplified single-subkey model misses this two-channel dependency.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.12 Carry ledger

The final addition in $\mathbb{Z}/2^{32}\mathbb{Z}$ converts a Boolean expression into a carry-sensitive word expression. XOR-linear trails do not pass through it unchanged.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.13 Lai-Massey correctness ledger

The F function can be non-invertible because the invariant $L \oplus R$ is preserved before and after F injection. This is the local reason decryption can recompute F.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.14 PHT inverse ledger

The PHT inverse solves a two-equation system over $\mathbb{Z}/2^{32}\mathbb{Z}$. The proof depends on modular addition and subtraction, not on XOR alone.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.15 Byte-substitution ledger

The forward and inverse schedules are fixed byte-wise permutations. Their correctness follows from table inversion and from applying inverse tables in the reverse schedule.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.16 RoundFunction work ledger

The data round is not one F call per macro-round. It is N^2 F-consuming word updates per macro-round, and 16 macro-rounds per block.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.17 Worker state ledger

The worker path is stateful. Correct decryption requires the same generated state trajectory; otherwise the inverse functions are applied under the wrong state.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.18 LFSR ledger

The LFSR is linear, so it is not a final security primitive. Its output is only one state material source consumed by later nonlinear layers.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.19 NLFSR ledger

The NLFSR contributes nonlinear gate-controlled feedback. Its value is in changing the matrix-update input, not in replacing the whole key schedule.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.20 SDP ledger

The double-pendulum source is deterministic state material. Its analysis boundary is explicit: the cipher does not rely on chaos alone as a proof of security.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.21 Quantum oracle ledger

A clean quantum oracle must compute, use, and uncompute the key schedule. The resource cost therefore includes the schedule, not only the final data round.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.22 Grover cost ledger

Grover reduces the number of key trials, but the cost per trial is $C_Q(\text{OPC})$. A heavy oracle remains heavy after quadratic query reduction.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.23 Simon model ledger

A Simon-style related-key attempt requires a hidden period in a derived function. The stateful key schedule makes constructing such a function a nontrivial modeling step.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.24 Differential-analysis ledger

A differential through the cipher crosses prime-field equations, sponge displacement, word carry, bit projection, table lookup, and Lai-Massey invariants. The trail model is multi-domain.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

20.25 Linear-analysis ledger

A linear approximation has to cross $\mathbb{F}_p$, $\mathbb{F}_2$, $\mathbb{Z}/2^{32}\mathbb{Z}$, and $\mathbb{Z}/2^{64}\mathbb{Z}$. Each crossing changes the meaning of the mask.

We use this ledger to keep the construction auditable. The local claim is intentionally small: it states a domain, an equation, an invariant, a resource cost, or a composition role. The whole-cipher statement is not obtained by exaggerating a local claim; it is obtained by composing many local claims with explicit boundaries.

For classical analysis, the ledger identifies which equation an adversary would have to solve or approximate. For quantum analysis, it identifies which reversible subcircuit or oracle component the adversary would have to build. This is why the paper treats complexity as structured cost rather than as decorative obscurity.

21 End-to-End Correctness and Implementation–Mathematics Correspondence

The end-to-end correctness argument is compositional. Packing and padding define the data representation. The key hardening layer defines the initial matrix-state material. Matrix update and round-subkey generation define the round material trajectory. The Lai–Massey word update is invertible because the XOR invariant allows recomputation of F . The PHT layer has an explicit inverse. The byte-substitution layer uses inverse tables. Decryption uses the same state trajectory and the reverse order of the data-round operations. Therefore, under identical initial state and key material, the encryption and decryption paths are inverse at the block level.

Proposition 21.1 (block-level correctness). Assume the same initial Type-2 state, the same master-key words, and the same IV/state parameters are used to generate the encryption and decryption trajectories. Then the decryption process applied to the encryption output recovers the original packed block sequence.

Proof. For each word update, Lai–Massey decryption inverts Lai–Massey encryption because $L \oplus R$ is preserved across the F injection and the PHT layer has an explicit inverse. For each byte-substitution layer, the inverse schedule uses inverse tables by definition. For each macro-round, decryption applies the inverse byte layer and then the inverse word updates in reverse key order. The composition of inverse maps in reverse order is the inverse of the encryption composition. Applying this argument to every macro-round and every block proves the claim. $\square$

22 Design Motivation, Analysis Boundary, and Future Work

We do not claim that the entire construction has a complete IND-CPA, PRP, SIS, or LWE reduction. We claim a precise set of local facts and design motivations. The SIS/Ajtai core gives a high-dimensional prime-field linear image whose short collision differences correspond to SIS-type relations. The TDOM sponge twists that image by a deterministic nonlinear displacement field. The 32-bit expansion and matrix schedule reshape the material before it reaches the data round. The Kronecker injection deliberately provides a collapsed directional state injection rather than a full-entropy random matrix. The BitMatrix gives bit-uniform parity distribution rather than MDS invertibility. The F function makes round behavior depend on state-related matrix addressing, masking, and modular carry. The Lai–Massey and byte-substitution layers provide reversible data transformation.

The next analysis tasks are therefore well defined: compute and bound differential trails through the F function under realistic state models; examine linear approximations that include the TDOM displacement field; evaluate algebraic degree growth through the 32-bit expansion and CrazyTransform; study state-collision behavior in matrix trajectories; and estimate quantum resource requirements under concrete reversible-circuit models rather than idealized free-oracle models.

23 Conclusion

This paper gives the English full-body technical specification of the `OaldresPuzzle_Cryptic` Type-2 block-cipher core. The construction combines an SIS/Ajtai-form prime-field hardening core, a white-box TDOM sponge, 32-bit nonlinear expansion,

stateful matrix update, Kronecker outer-product injection, non-commutative round-subkey generation, key-state whitening, Bit-Matrix projection, a state-dependent F function, Lai–Massey word permutations, static byte substitution, and a stateful worker path. We describe the algebraic spaces separately, provide default parameter sizes, give quantum gate-cost accounting, and identify both provable local properties and explicit analysis boundaries. The resulting document is not a claim of magical security; it is a structured specification and analysis foundation for a deliberately hard-to-model stateful symmetric construction.

References

- [1] Fırat Artuğer and Fatih Özkaynak. A method for generation of substitution box based on random selection. *Egyptian Informatics Journal*, 23(1):127–135, 2022.
- [2] Fabio Borges, Paulo Ricardo Reis, and Diogo Pereira. A comparison of security and its performance for key agreements in post-quantum cryptography. *IEEE Access*, 8:142413–142422, 2020.
- [3] Amit Kumar Chauhan and Somitra Sanadhya. Quantum security of fox construction based on lai-massey scheme. Cryptology ePrint Archive, Paper 2022/1001, 2022. <https://eprint.iacr.org/2022/1001>.
- [4] Guillermo Cotrina, Alberto Peinado, and Andrés Ortiz. Gaussian pseudorandom number generator using linear feedback shift registers in extended fields. *Mathematics*, 9(5):556, 2021.
- [5] Aarti Dadheech. Post-quantum lattice-based cryptography: A quantum-resistant cryptosystem. In *Limitations and Future Applications of Quantum Cryptography*, pages 102–123. IGI Global, 2021.
- [6] Joan Daemen and Vincent Rijmen. *The design of Rijndael*, volume 2. Springer, 2002.
- [7] Muhammed Fethullah Esgin. *Practice-oriented techniques in lattice-based cryptography*. PhD thesis, Monash University, 2020.
- [8] Oded Goldreich, Shafi Goldwasser, and Shai Halevi. Collision-free hashing from lattice problems. *IACR Cryptol. ePrint Arch.*, 1996:9, 1996.
- [9] Marc Kaplan, Gaëtan Leurent, Anthony Leverrier, and María Naya-Plasencia. Breaking symmetric cryptosystems using quantum period finding. In *Advances in Cryptology – CRYPTO 2016*, pages 207–237. Springer Berlin Heidelberg, 2016.
- [10] Kazys Kazlauskas and Jaunius Kazlauskas. Key-dependent s-box generation in aes block cipher system. *Informatica, Lith. Acad. Sci.*, 20:23–34, 01 2009.
- [11] Hidenori Kuwakado and Masakatu Morii. Quantum distinguisher between the 3-round feistel cipher and the random permutation. In *2010 IEEE International Symposium on Information Theory*, pages 2682–2685, 2010.
- [12] Hidenori Kuwakado and Masakatu Morii. Security on the quantum-type even-mansour cipher. In *2012 International Symposium on Information Theory and its Applications*, pages 312–316, 2012.
- [13] Yiyuan Luo, Xuejia Lai, and Yujie Zhou. Generic attacks on the lai–massey scheme. *Designs, Codes and Cryptography*, 83, 05 2017.
- [14] Ashwini Kumar Malviya, Namita Tiwari, and Meenu Chawla. Quantum cryptanalytic attacks of symmetric ciphers: A review. *Computers and Electrical Engineering*, 101:108122, 2022.
- [15] Shuping Mao, Tingting Guo, Peng Wang, and Lei Hu. Quantum attacks on lai-massey structure. Cryptology ePrint Archive, Paper 2022/986, 2022. <https://eprint.iacr.org/2022/986>.
- [16] Chandra Sekhar Mukherjee, Dibyendu Roy, and Subhamoy Maitra. Design specification of zuc stream cipher. In *Design and Cryptanalysis of ZUC*, pages 43–62. Springer, 2021.
- [17] Chokri Nouar and Z. Guennoun. A pseudo-random number generator using double pendulum. *Applied Mathematics & Information Sciences*, 14:977–984, 11 2020.
- [18] Stjepan Picek, Lejla Batina, Domagoj Jakobović, Bariş Ege, and Marin Golub. S-box, SET, Match: A Toolbox for S-box Analysis. In David Naccache and Damien Sauveron, editors, *8th IFIP International Workshop on Information Security Theory and Practice (WISTP)*, volume LNCS-8501 of *Information Security Theory and Practice. Securing the Internet of Things*, pages 140–149, Heraklion, Crete, Greece, June 2014. Springer. Part 5: Short Papers.
- [19] Richard Preston. Applying grover’s algorithm to hash functions: A software perspective, 2022.

- [20] Tomasz Rachwalik, Janusz Szmidt, Robert Wicik, and Janusz Zablocki. Generation of nonlinear feedback shift registers with special-purpose hardware. Cryptology ePrint Archive, Paper 2012/314, 2012. <https://eprint.iacr.org/2012/314>.
- [21] Maximilian Richter, Magdalena Bertram, Jasper Seidensticker, and Alexander Tschache. A mathematical perspective on post-quantum cryptography. *Mathematics*, 10(15), 2022.
- [22] Ronald L Rivest, Matthew JB Robshaw, Ray Sidney, and Yiqun Lisa Yin. The rc6tm block cipher. In *First advanced encryption standard (AES) conference*, page 16, 1998.
- [23] M. R. Mirzaee Shamsabad and S. M. Dehnavi. Lai-massey scheme revisited. Cryptology ePrint Archive, Paper 2020/005, 2020. <https://eprint.iacr.org/2020/005>.
- [24] D.R. Simon. On the power of quantum computation. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 116–123, 1994.
- [25] Nigel P Smart and Emmanuel Thom  . History of Cryptographic Key Sizes. In Joppe Bos and Martijn Stam, editors, *Computational Cryptography*, volume 469 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, December 2021.
- [26] Han SUI and Wenling WU. Research status and development trend of post-quantum symmetric cryptography. *Journal of Electronics & Information Technology*, 42(190667):287, 2020.
- [27] Kethan Vooke, Nithin Kumar Toramamidi, Kalyan Kumar Thodeti, and Sangeeta Singh. Design of pseudo-random number generator using non-linear feedback shift register. In *2022 First International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*, pages 1–5, 2022.
- [28] Zeguo Wang, Shijie Wei, Gui Long, and L. Hanzo. Variational quantum attacks threaten advanced encryption standard based symmetric cryptography. *Science China Information Sciences*, 65:200503, 10 2022.
- [29] Aaram Yun, Je Hong Park, and Jooyoung Lee. Lai-massey scheme and quasi-feistel networks. Cryptology ePrint Archive, Paper 2007/347, 2007. <https://eprint.iacr.org/2007/347>.